

STA 완전이해 — Static Timing Analysis를 1st-principles로

🔥 이 글의 목적: STA를 "공식을 외우는 과목"이 아니라 **하나의 물리적 약속(동기 설계 계약)에서 모든 것이 따라 나오는 체계**로 다시 이해한다. 당신은 이미 삼성 S.LSI에서 PrimeTime으로 timing closure를 직접 돌려 본 사람이다. 그래서 이 글은 "setup이 뭐냐"를 처음 가르치는 글이 아니라, **이미 손에 익은 감각을 1st-principles로 말로 풀어낼 수 있게** 만드는 글이다. 면접관이 "왜요?"를 세 번 던져도 무너지지 않도록.

🧠 메모리 브릿지의 핵심: 당신이 Peri/Base Die 로직에서 하던 timing closure는 그대로 가치가 있고, 거기에 더해 **DDR/HBM의 데이터-스트로브(DQ-DQS) 타이밍**이라는, STA가 가장 극한으로 작동하는 무대가 기다린다. 10GT/s, 2048-bit 광폭 I/O에서는 setup/hold 마진 picosecond가 곧 수율이고 곧 제품 등급이다.

0. 들어가며 — 왜 STA인가, sign-off란 무슨 뜻인가

칩을 만들면 "이게 1GHz에서 제대로 동작하느냐"를 보장해야 한다. 가장 소박한 방법은 **시뮬레이션**이다. 입력 벡터를 넣고, 게이트 지연을 반영해서, 출력이 맞는지 본다. 그런데 이 방법에는 치명적인 약점이 있다.

첫째, **벡터가 필요하다**. 회로의 어떤 경로가 가장 느린지 미리 알 수 없으니, 그 경로를 자극하는 입력 패턴을 누군가 만들어 줘야 한다. 수백만 개 플립플롭과 그 사이의 조합논리가 만드는 경로의 수는 천문학적이다. 그 전부를 자극하는 벡터 집합을 만든다는 것은 사실상 불가능하다. 빠뜨린 경로가 하필 가장 느린 경로라면, 시뮬레이션은 "통과"라고 말하지만 실리콘은 죽는다.

둘째, **느리다**. 게이트 레벨 타이밍 시뮬레이션은 한 사이클을 도는 데도 막대한 계산이 든다. 큰 칩을 모든 코너에서 충분한 길이로 돌리는 것은 시간이 허락하지 않는다.

STA(Static Timing Analysis, 정적 타이밍 분석)는 이 두 약점을 정면으로 푼다. **벡터가 필요 없다(vectorless)**. STA는 입력을 넣어 회로를 "돌리는" 것이 아니라, 회로 그래프를 펼쳐 놓고 **모든 타이밍 경로의 지연을 정적으로 계산**한다. launch 플립플롭에서 출발해 조합논리를 거쳐 capture 플립플롭에 도착하기까지, 각 셀과 배선의 지연을 더해서 "이 경로는 최악의 경우 몇 ps가 걸린다"를 산출한다. 그리고 그 경로 전부를 **빠짐없이(exhaustive)** 본다. 어떤 패턴을 안 넣어서 놓치는 일이 구조적으로 발생하지 않는다. 게다가 시뮬레이션보다 비교할 수 없이 **빠르다** — 회로를 돌리는 게 아니라 그래프 위에서 덧셈을 하는 것이기 때문이다.

이 "벡터 없이, 빠짐없이, 빠르게"라는 세 성질 때문에 STA는 **sign-off**의 핵심이 된다. sign-off란 "이 설계는 명세된 조건(주파수·전압·온도·공정 변이)에서 타이밍을 만족함을 보증한다"고 **서명**하는 행위다. 테이프아웃(마스크 제작, 즉 되돌릴 수 없는 단계)으로 넘어가기 직전, "정말 동작하는가?"에 대한 최종 책임 있는 답이다. 산업에서 이 서명을 받는 도구의 사실상 표준이 Synopsys PrimeTime이고, 당신이 손에 익힌 바로 그 도구다. PrimeTime은 여기에 crosstalk에 의한 지연 변화(SI), 통계적 변이(POCV) 같은 정밀 효과까지 얹어 마지막 마진을 가른다.

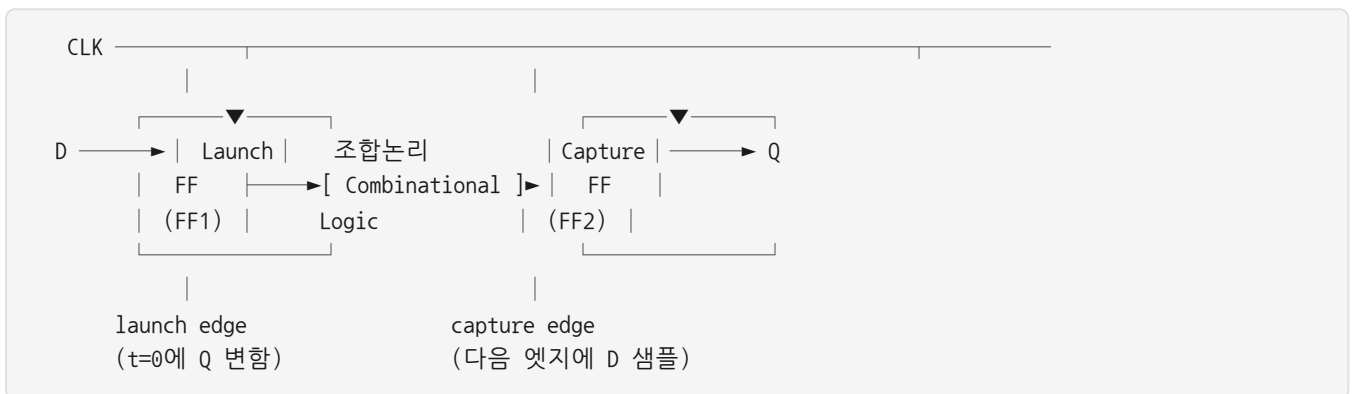
한 문장 요약: STA는 "벡터 없이 모든 경로를 정적으로 계산해 setup/hold를 빠짐없이 검증"하는, 테이프아웃 전 최종 서명(sign-off)의 도구다.

1. 동기 설계의 약속 — 모든 것이 여기서 시작한다

STA를 1st-principles로 이해하는 출발점은 동기 설계(synchronous design)가 무엇을 약속하는가이다. 이 약속을 정확히 붙들면, setup도 hold도 skew도 전부 거기서 연역적으로 따라 나온다.

동기 설계의 세계관은 단순하다. 회로에는 클럭이라는 공통의 박자가 있고, 모든 상태(state)는 플립플롭(FF)에 담긴다. 데이터는 클럭의 엣지(보통 상승 엣지)에 맞춰 한 FF에서 다음 FF로 "한 칸씩" 전진한다. 그 사이에는 조합논리(AND/OR/MUX/덧셈기 등, 상태가 없는 논리)가 있어서, 한 FF가 내보낸 값을 가공해 다음 FF의 입력으로 보낸다.

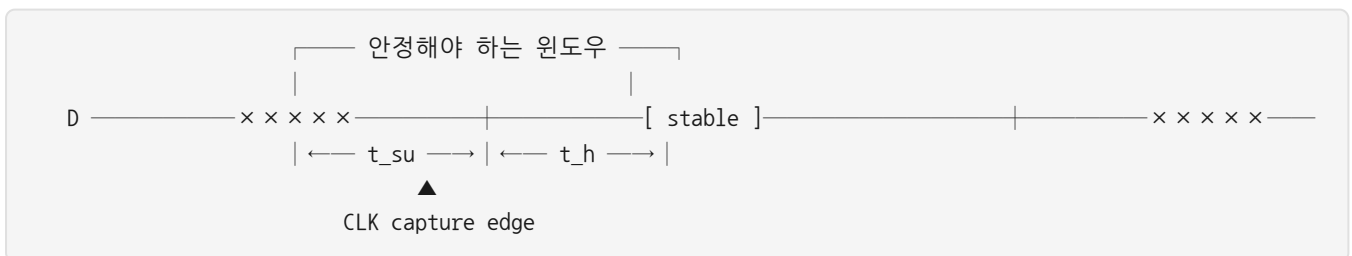
그림으로 박아 두자. 이것이 STA가 보는 타이밍 경로(timing path)의 기본 단위다.



여기서 한 경로는 항상 "출발 FF(launch) → 조합논리 → 도착 FF(capture)"의 3박자로 이루어진다. STA가 검증하는 것은 단 하나의 계약이다:

launch FF가 한 클럭 엣지에 데이터를 내보내면, 그 데이터는 조합논리를 통과해 capture FF가 "안전하게 받을 수 있는 시점"에 도착해야 한다.

"안전하게 받는다"는 것이 핵심인데, 플립플롭은 입력 D가 클럭 엣지 주변의 일정 구간 동안 흔들리지 않고 안정(stable)해야만 그 값을 정확히 포획한다. 이 안정 구간은 두 부분으로 나뉜다. 엣지 이전(before)에 필요한 안정 시간이 setup time(t_{su}), 엣지 이후(after)에 필요한 안정 시간이 hold time(t_h)이다.



왜 setup과 hold가 둘 다 필요한가? 플립플롭 내부를 들여다보면 답이 보인다. 엣지 직전에는 입력을 마스터 래치에 "퍼 담는" 동작이 일어나고, 입력이 그때까지 안정해야 마스터가 올바른 값을 잡는다(→ setup). 엣지 직후에는 마스터에서 슬레이브로 값이 "넘어가 잠기는" 동안 입력단이 아직 닫히기 전이라, 너무 일찍 입력이 바뀌면 방금 잡은 값을 덮어쓸 수 있다(→ hold). 즉 setup은 "충분히 일찍 와라", hold는 "너무 일찍 떠나지 마라"는 두 개의 독립된 요구다. 이 두 요구가 STA의 두 기동, setup 분석과 hold 분석이 된다.

만약 이 안정 윈도우를 침범하면? 플립플롭은 0도 1도 아닌 어중간한 전압에 갇히는 **메타스터빌리티 (metastability)**에 빠질 수 있다. 출력이 한동안 0.5Vdd 근처에서 진동하다가 한참 뒤 임의의 값으로 떨어진다. 그 "한참"이 다음 단의 setup을 또 잡아먹으면 오류가 전파된다. STA의 존재 이유는 결국, **모든 경로에서 이 안정 윈도우가 절대 침범되지 않음을 사전에 보증하는 것이다.**

기억할 한 문장: STA는 "launch→logic→capture 한 칸 전진"이라는 동기 설계의 약속이 **모든 경로에서, 모든 조건에서** 지켜지는지를 검사한다. setup은 '엣지 전 안정', hold는 '엣지 후 안정'이라는 두 요구로 갈린다.

2. Setup 분석 — "다음 엣지 전에 도착하라"

이제 첫 번째 기동, setup을 1st-principles로 세워 보자. setup의 직관은 이렇다:

launch 엣지에 출발한 데이터는, 그 다음 capture 엣지가 오기 전에 (정확히는 t_{su} 만큼 일찍) 도착해 안정해 있어야 한다.

핵심은 "**다음(next) 엣지**"라는 점이다. 데이터는 launch 엣지에 출발해서, 한 클럭 주기(T_{clk}) 뒤에 오는 capture 엣지에 잡힌다. 그러니 데이터에게 주어진 시간 예산은 본질적으로 **한 클럭 주기**다. 그 예산 안에 다음 일이 다 끝나야 한다:

1. launch FF가 클럭을 받고 출력 Q를 내보내는 데 걸리는 시간 — t_{cq} (clock-to-Q delay)
2. 그 Q가 조합논리를 통과하는 데 걸리는 시간 — t_{comb} (logic delay)
3. capture FF의 D에 도착해서 t_{su} 만큼 미리 안정해 있어야 한다는 요구 — t_{su}

이걸 부등식으로 쓰면 setup 계약이 된다 (skew는 잠시 0으로 두자):

$$\underbrace{t_{cq} + t_{comb} + t_{su}}_{\text{도착(arrival)}} \leq T_{clk} \quad \underbrace{\hspace{1.5cm}}_{\text{요구(required)}}$$

좌변은 "데이터가 실제로 안정해 있을 수 있는 가장 늦은 시각(에 t_{su} 를 더한 것)"이고, 우변은 "다음 엣지까지 허용된 시간"이다. 이 부등식의 여유분을 **setup slack**이라 부른다:

$$\begin{aligned} \text{Setup slack} &= T_{clk} - (t_{cq} + t_{comb} + t_{su}) \\ &= (\text{요구 시간}) - (\text{도착 시간}) \end{aligned}$$

slack이 0 이상이면 만족, 음수면 위반이다. 여기서 **arrival time(도착 시간)**과 **required time(요구 시간)**이라는 STA의 두 핵심 어휘가 등장한다. arrival은 "데이터가 실제로 capture FF의 D에 도착하는 시각", required는 "도착해도 되는 가장 늦은 시각(t_{su} 를 뺀 마감 시한)"이다. **slack = required - arrival** — 이 한 줄이 STA 전체를 관통한다.

직관을 더 밀어 보자. setup 위반은 **데이터가 너무 느려서** 생긴다. 조합논리가 길거나(많은 게이트), 게이트 하나하나가 느리거나(약한 셀, 큰 부하), 배선이 멀어서 t_{comb} 이 커지면, 도착이 마감보다 늦어진다. 그래서

setup을 결정하는 것은 **가장 느린 경로(max delay path)**다. 이 가장 느린 경로, slack이 가장 작은(가장 음수에 가까운) 경로를 **critical path**라 부른다. 칩의 최대 동작 주파수를 결정하는 바로 그 경로다.

여러 경로 중 setup을 위협하는 건 '가장 느린' 경로:

```
path A:  t_cq + t_comb_A + t_su = 0.9 ns    (slack +0.1, T=1.0ns)
path B:  t_cq + t_comb_B + t_su = 1.05 ns   (slack -0.05) ← critical, 위반!
path C:  t_cq + t_comb_C + t_su = 0.6 ns    (slack +0.4)
```

setup은 가장 긴 path B가 결정한다 (worst = max delay)

이제 면접에서 반드시 나오는 질문에 1st-principles로 답할 수 있다: **"setup 위반은 왜 주파수를 낮추면 해결되는가?"**

setup slack 식을 다시 보라. $\text{Setup slack} = T_{\text{clk}} - (\text{도착})$. 도착 시간($t_{\text{cq}} + t_{\text{comb}} + t_{\text{su}}$)은 회로의 물리적 성질이라 고정이다. 그런데 T_{clk} 는 주파수의 역수다. 주파수를 낮추면 T_{clk} 이 커지고, 우변이 통째로 커지니 slack이 양수 쪽으로 밀려난다. 즉 **데이터에게 더 긴 시간 예산을 주는 것이다**. 느린 경로라도 충분히 기다려 주면 도착할 수 있다. 그래서 setup 위반은 "주파수를 낮춰서 일단 동작은 시키는" 임시 회피가 가능하다. 물론 그건 성능을 깎는 것이고, 제품으로는 그 주파수가 곧 등급(speed bin)이 된다. 메모리로 치면 "이 칩은 1600 대신 1333에서만 통과"가 되는 셈이다. 본질적 수정은 결국 **경로를 빠르게 만드는 것** — 셀 키우기, VT 낮추기, 로직 단순화, 배선 줄이기다.

기억할 한 문장: setup = "다음 엣지 전 도착". slack = $T_{\text{clk}} - (t_{\text{cq}} + t_{\text{comb_max}} + t_{\text{su}})$. **느린(max) 경로가 critical, 주파수를 낮추면($T_{\text{clk}} \uparrow$) 풀린다** — 데이터에게 시간을 더 주는 것이기 때문.

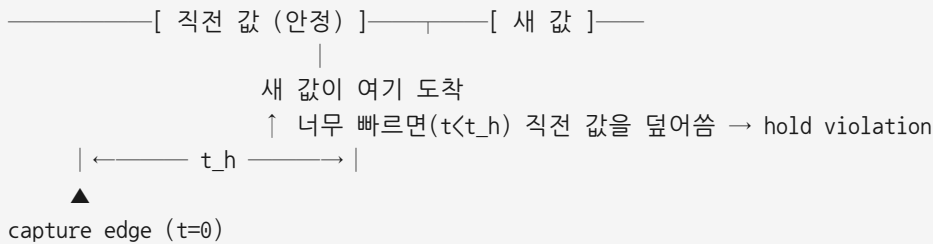
3. Hold 분석 — "같은 엣지 후에도 잠깐 머물러라"

두 번째 기둥, hold는 처음 보면 헷갈리지만 1st-principles로 따라가면 명료하다. setup이 "다음 엣지"의 문제였다면, hold는 **"같은(same) 엣지"**의 문제다. 이 한 단어의 차이가 모든 것을 가른다.

상황을 그려 보자. launch FF와 capture FF가 **같은 클럭 엣지**를 받는다. 그 엣지 순간, capture FF는 자신의 D에 들어와 있는 (직전 사이클의) 값을 잡으려 한다. 그런데 같은 엣지에 launch FF도 새 값을 내보낸다. 만약 launch가 내보낸 **새 값이 너무 빨리** 조합논리를 통과해 capture FF의 D에 도달해 버리면, capture FF가 **방금 잡았어야 할 옛 값을 채 잠그기도 전에** 새 값이 덮어써 버린다. 결과는 "한 사이클을 건너뛰는" 데이터 경합 — race다.

같은 capture 엣지 $t=0$ 에서:

capture FF는 '직전 값'을 잡아야 함 (이 값을 t_h 동안 지켜줘야)



그래서 hold의 계약은 이렇다:

capture 엣지 직후 t_h 동안, capture FF의 D는 흔들리면 안 된다. 즉 launch가 같은 엣지에 내보낸 새 값은 적어도 t_h 가 지난 뒤에 도착해야 한다.

부등식으로 (skew=0):

$$\underbrace{t_{cq} + t_{comb(min)}}_{\text{새 값의 도착}} \geq t_h \quad \text{「 유지 요구 」}$$

여기서 t_{comb} 이 min인 것에 주목하라. hold를 위협하는 것은 **가장 빠른 경로(min delay path)**다. setup이 "가장 느린 경로가 마감을 못 맞출까 봐" 걱정했다면, hold는 정반대로 **"가장 빠른 경로가 너무 빨리 도착해 옛 값을 깔까 봐"** 걱정한다. hold slack은:

$$\begin{aligned} \text{Hold slack} &= (t_{cq} + t_{comb_min}) - t_h \\ &= (\text{도착}) - (\text{요구}) \end{aligned}$$

(주의: setup slack은 required-arrival인데 hold slack은 arrival-required다. 부호가 뒤집힌다. hold는 "충분히 늦게 도착했는가"를 보므로 도착이 클수록 좋다. 이 부호 감각이 면접에서 skew 질문을 가를 때 결정적이다.)

이제 setup과 짝을 이루는 결정적 질문: **"hold 위반은 왜 주파수와 무관한가?"**

hold slack 식을 노려보라. $\text{Hold slack} = (t_{cq} + t_{comb_min}) - t_h$. 이 식 어디에도 T_{clk} 이 없다. 당연하다 — hold는 launch와 capture가 **같은 엣지**를 공유하는 문제이지, "다음 엣지까지 얼마나 남았나"의 문제가 아니다. 두 엣지 사이의 간격(=주기)이 얼마든, "같은 엣지에서 새 값이 옛 값을 너무 빨리 덮느냐"는 그 간격과 아무 상관이 없다. 그래서 **주파수를 아무리 낮춰도 hold는 한 톨도 좋아지지 않는다**. clock을 늦추는 것은 hold에게 무의미하다.

이것이 hold 위반이 setup 위반보다 **훨씬 위험한** 이유다. setup 위반은 최악의 경우 주파수를 낮춰 제품 등급을 깎아서라도 살릴 수 있지만, hold 위반은 **실리콘에서 발견되면 그 칩은 어떤 주파수에서도 죽는다(dead)**. 동작 자체가 불가능하다. 메모리처럼 수억 개를 양산하는 제품에서 hold 위반이 마스크에 박혀 나오면 그건 곧 전량 폐기다.

그럼 hold는 어떻게 고치는가? 본질은 "가장 빠른 경로를 일부러 느리게" 만드는 것이다. 가장 흔한 방법이 **버퍼(또는 delay cell) 삽입** — 빠른 경로 중간에 지연 소자를 끼워 t_{comb_min} 을 키운다. 그러면 새 값의 도착이 늦춰져 옛 값을 지켜 줄 시간이 생긴다.

그런데 이게 **까다롭다**. 두 가지 이유가 있다. 첫째, 버퍼를 넣어 hold를 고치면 **그 경로의 setup이 동시에 나빠진다**. 경로가 느려졌으니 setup slack을 깎아먹는다. setup 여유가 빠듯한 경로에서는 hold를 고치다가 setup을 깨는 줄타기가 벌어진다. 둘째, **거의 모든 코너에서 동시에** hold가 만족돼야 한다 — 뒤에서 볼 fast 코너에서 가장 빠른 경로가 더 빨라지므로, 한 코너에서만 고쳐서는 안 된다. 게다가 hold 위반 경로는 칩 전체에 수만 개씩 흩어져 있을 수 있어, 버퍼 삽입 ECO가 면적·배선을 잠식한다. 그래서 hold closure는 setup closure보다 손이 더 가고, 실수 여지가 크다.

기억할 한 문장: hold = "같은 엣지 후 유지". slack = $(t_{cq} + t_{comb_min}) - t_h$, 식에 T_{clk} 이 없다 → 주파수 무관. 빠른(min) 경로가 위험, 버퍼 삽입으로 고치되 setup·다중 코너 때문에 까다롭다. 위반 시 chip dead.

4. Slack의 부호와 critical path — 한 줄로 꿰기

앞에서 흩어진 slack 개념을 한자리에 모아 뜻을 박자. STA의 모든 보고서는 결국 **slack** 하나로 환원된다.

$slack = required\ time - arrival\ time$ (setup 관점의 표준 정의)

$slack > 0$ → 여유 있음, 타이밍 만족 (met)

$slack = 0$ → 딱 맞음 (marginal)

$slack < 0$ → 위반 (violation)

setup에서는 arrival을 줄이거나 required를 늘리면(주파수↓) slack이 좋아진다. hold에서는 부호가 뒤집혀, **arrival을 키우면(경로를 느리게)** slack이 좋아진다 — 그래서 hold slack은 $arrival - required$ 로 쓰는 게 직관적이다. 이 부호의 비대칭을 머릿속에 새겨 두면, "어떤 조작이 setup엔 좋고 hold엔 나쁜가"가 자동으로 풀린다. 경로를 빠르게 하면 $setup↑·hold↓$, 느리게 하면 $setup↓·hold↑$. 이 시소가 timing closure의 모든 줄타기의 근원이다.

칩 전체로 보면 두 개의 집계 지표가 중요하다. **WNS(Worst Negative Slack)**는 가장 나쁜 한 경로의 slack — 얼마나 심하게 깨졌나(깊이). **TNS(Total Negative Slack)**는 음수 slack을 모두 더한 값 — 얼마나 많은 경로가 깨졌나(규모). WNS만 보면 "최악 경로 하나가 -50ps"라는 깊이는 알지만, 그게 1개인지 1만 개인지는 모른다. closure의 난이도와 ECO 규모를 가늠하려면 **WNS와 TNS, 그리고 위반 경로 수를 함께** 봐야 한다. 면접에서 "WNS만 보면 되나요?"라는 꼬리질문이 나오면, "아니요, TNS와 위반 경로 수까지 봐야 ECO 물량을 안다"가 정답이다.

critical path는 slack이 가장 작은(가장 음수에 가까운) 경로다. setup critical path는 칩의 최대 주파수를 결정하고, 그것을 빠르게 만드는 것이 성능 향상의 직접적 지렛대다. closure 작업의 상당 부분이 "critical path를 찾아 한 단계씩 빠르게"의 반복이다.

5. Clock skew — 같은 클럭이 다른 시각에 도착할 때

지금까지 우리는 launch와 capture가 클럭을 **동시에** 받는다고 가정했다(skew=0). 현실은 다르다. 클럭은 클럭 트리(clock tree)를 타고 칩 전역에 퍼지는데, FF마다 위치가 다르고 배선 길이·버퍼 단수가 달라 **클럭 엣지가**

도착하는 시각이 FF마다 미세하게 다르다. 이 launch FF와 capture FF 사이의 클럭 도착 시각 차이가 **clock skew**다.

정의를 부호까지 못 박자:

$$\text{skew} = (\text{capture FF의 클럭 도착 시각}) - (\text{launch FF의 클럭 도착 시각})$$

capture 클럭이 launch보다 **늦게** 도착하면 $\text{skew} > 0$ 이다. 이게 setup과 hold에 **정반대 부호**로 작용한다는 게 핵심이고, 면접 단골이다.

setup에 미치는 영향. capture 엣지가 늦게 온다($\text{skew} > 0$)는 건, 데이터에게 **마감이 그만큼 미뤄진**다는 뜻이다. 즉 시간을 더 받는다. setup 부등식에 넣으면:

$$\begin{aligned} t_{cq} + t_{comb} + t_{su} &\leq T_{clk} + \text{skew} \\ \text{Setup slack} &= (T_{clk} + \text{skew}) - (t_{cq} + t_{comb_max} + t_{su}) \end{aligned}$$

skew가 양수면 setup slack이 커진다 → setup에 **유리**. (capture를 일부러 늦추는 "useful skew"가 critical path의 setup을 살리는 기법이 여기서 나온다.)

hold에 미치는 영향. 그런데 같은 $\text{skew} > 0$ (capture가 늦게 옴)이 hold에는 **독이 된다**. hold는 "같은 엣지에서 capture가 옛 값을 잡는 동안 새 값이 너무 빨리 오면 안 된다"는 문제였다. capture 엣지가 **늦게** 온다는 건, capture FF가 옛 값을 잡는 시점 자체가 뒤로 밀린다는 뜻이고, 그동안 launch는 (자기 엣지에 맞춰) 이미 새 값을 일찍 내보내 버린다. 즉 새 값이 capture의 (늦어진) 포획 시점을 침범하기가 더 쉬워진다. hold 부등식에 넣으면:

$$\begin{aligned} t_{cq} + t_{comb(min)} &\geq t_h + \text{skew} \\ \text{Hold slack} &= (t_{cq} + t_{comb_min}) - (t_h + \text{skew}) \end{aligned}$$

skew가 양수면 hold slack이 **작아진다** → hold에 **불리**. 부호가 setup과 정확히 반대로 들어간다.

$$\begin{aligned} \text{skew} > 0 \text{ (capture 클럭이 늦게 도착):} \\ \text{setup slack: } (T_{clk} + \text{skew}) - \text{arrival} &\rightarrow \text{skew가 더해짐} \rightarrow \text{유리} \uparrow \\ \text{hold slack: } \text{arrival} - (t_h + \text{skew}) &\rightarrow \text{skew가 빠짐} \rightarrow \text{불리} \downarrow \\ &\quad \quad \quad \blacktriangle\blacktriangle\blacktriangle \\ &\quad \quad \quad \text{같은 skew, 정반대 부호 — 이것이 skew의 본질} \end{aligned}$$

그래서 "**skew가 hold에 왜 위험한가**"에 대한 1st-principles 답은: capture 클럭이 늦게 도착하면 capture FF의 포획 시점이 뒤로 밀려 hold 윈도우가 사실상 넓어지고, launch의 빠른 새 값이 그 넓어진 윈도우를 침범하기 쉬워지기 때문이다. 더 직관적으로는, **skew가 t_h 를 키운 것과 같은 효과를 낸다** — hold 요구가 더 빡빡해진다. 빠른 경로(짧은 t_{comb_min})에서 skew가 크면 hold가 무너진다. 그래서 클럭 트리를 잘 균형 잡아(skew 최소화) 짓는 CTS가 hold closure의 사전 작업으로서 결정적이다.

기억할 한 문장: $\text{skew} = \text{capture클럭} - \text{launch클럭}$ 도착 시각 차. **setup엔 +로(유리), hold엔 -로(불리)** 작용 — 부호가 반대다. capture가 늦으면 setup은 시간을 벌지만 hold 윈도우가 넓어져 빠른 경로가 무너진다.

6. Jitter · uncertainty · OCV · PVT corner — 불확실성을 보수적으로 끌어안기

skew가 정적(static) 인 클럭 도착 시각 차이라면, jitter는 동적(dynamic) 변동이다. 같은 지점에서도 클럭 엣지가 매 사이클 약간씩 흔들린다(앞당겨지거나 늦춰진다). 원인은 PLL/DLL 내부 노이즈, 전원 노이즈, crosstalk 등이다. jitter는 매 사이클 예측 불가능하게 엣지를 떨게 하므로, STA는 이를 **마진을 깎는 불확실성**으로 모델링한다.

STA는 skew·jitter·여유 마진을 묶어 **clock uncertainty**라는 하나의 보수적 양으로 다룬다. 그리고 핵심 원리는 **항상 worst를 가정하는 것**이다:

- **setup 체크**: uncertainty를 **빼서**(available time을 줄여서) 본다. "운 나쁘게 capture 엣지가 일찍 와서 시간이 줄었다면?"
- **hold 체크**: uncertainty를 **더해서**(요구를 키워서) 본다. "운 나쁘게 hold 윈도우가 더 넓어졌다면?"

```
setup: arrival ≤ (T_clk + skew) - uncertainty   (불확실성만큼 마감을 당겨 본다)
hold:  arrival ≥ (t_h + skew) + uncertainty     (불확실성만큼 요구를 늘려 본다)
```

여기서도 setup엔 -, hold엔 +라는 같은 pessimism 부호 패턴이 반복된다. STA의 일관된 철학 — **불리한 쪽으로 가정해서, 통과하면 진짜로 안전하다**.

다음은 **OCV(On-Chip Variation)**와 **derating**이다. 같은 칩 안에서도 위치마다 공정 미세 편차, 국소 전압 강하, 온도 분포가 달라, **물리적으로 동일한 셀이라도 지연이 다르다**. 이 변이를 STA가 무시하면 낙관적 결과가 나온다. 그래서 STA는 경로에 **derating factor**를 곱해 변이를 끌어안는다. setup 체크에서는 데이터 경로(launch path)는 느리게, 클럭 경로(capture clock path)는 빠르게 가정해 **최악의 조합**을 본다. hold 체크에서는 반대로 둔다. 단순 OCV는 모든 셀에 같은 derate를 일괄 적용해 과도하게 비관적(pessimistic)이 되기 쉽다. 경로가 깊을수록 변이가 평균화돼 상쇄되는데도 그걸 무시하기 때문이다. 이를 완화하려고 경로 깊이(stage 수)에 따라 derate를 줄이는 **AOCV(Advanced OCV)**, 셀별 지연 분포를 통계적으로 다루는 **POCV(Parametric/statistical OCV)**가 쓰인다. PrimeTime의 sign-off가 POCV를 쓰는 이유가 바로 "현실적이면서도 안전한 마진"을 얻기 위함이다.

마지막으로 **PVT corner**다. PVT는 Process(공정)·Voltage(전압)·Temperature(온도)의 약자다. 같은 설계라도 공정이 빠르게 빠진 칩(FF)과 느리게 빠진 칩(SS)이 있고, 전압이 높거나 낮을 수 있고, 온도가 고온이거나 저온일 수 있다. 이 조합 각각에서 지연이 달라진다. STA는 **여러 코너에서 모두** 타이밍을 만족함을 봐야 한다 — 어느 한 코너라도 깨지면 그 조건의 칩은 죽기 때문이다. 이것이 **"왜 여러 코너를 보나"**의 답이다: 양산되는 수억 개 칩이 모두 같지 않고, 동작 환경(전압·온도)도 천차만별이라, 그 분포의 양 극단을 다 막아야 한다.

직관적 짝을 외워 두자:

```
Setup worst → Slow corner   (SS 공정, 저전압, (대개) 고온)
→ 셀이 느려 t_comb_max가 커짐 → 도착이 늦어짐 → setup이 깨지기 쉬움
Hold worst → Fast corner    (FF 공정, 고전압, (대개) 저온)
→ 셀이 빨라 t_comb_min이 작아짐 → 새 값이 너무 빨리 도착 → hold가 깨지기 쉬움
```

(온도의 부호는 노드에 따라 "temperature inversion"으로 역전될 수 있어 단정은 피하되, **setup은 느린 쪽, hold는 빠른 쪽**이라는 큰 그림은 견고하다.) 메모리는 동작 전압·온도 범위가 넓어 코너 수가 많고, 특히 **저온·고전압의 fast 코너에서 hold 마진** 관리가 까다롭다. 면접에서 "OCV/corner 왜 보나?"가 나오면, "한 칩·한 조건이 아니라 양산 분포 전체와 동작 환경 전체에서 slack≥0을 보장해야 하고, OCV는 한 칩 안의 국소 변이까지 끌어안기 위함"이라고 1st-principles로 답하면 된다.

기억할 한 문장: jitter=동적 엣지 떨림, uncertainty=skew+jitter+마진(setup엔 -, hold엔 +). OCV=칩 내부 변이를 derate로, AOCV/POCV로 과도 pessimism 완화. PVT corner는 양산·환경 분포의 양 극단 — setup=slow corner, hold=fast corner.

7. 제약(SDC) — STA에게 "무엇을, 어떻게 보라"고 알려 주기

STA는 강력하지만, 회로의 의도를 스스로 알지는 못한다. "클럭 주기가 얼마인지", "입력이 언제 도착하는지", "이 경로는 사실 타이밍을 안 잡아도 되는지"를 사람이 알려 줘야 한다. 그 명세 언어가 **SDC(Synopsys Design Constraints)**다. SDC를 잘못 쓰면 STA가 엉뚱한 것을 검증하므로, SDC는 STA의 신뢰성을 떠받치는 입력이다. 핵심 명령 몇 가지를 1st-principles로 보자.

create_clock — 클럭을 정의한다. 주기(T_clk), 파형(상승/하강 시점), 소스 핀을 지정한다. 이게 없으면 STA는 "박자"가 무엇인지 모른다. setup의 "다음 엣지", hold의 "같은 엣지"가 전부 이 클럭 정의에서 나온다. 모든 타이밍 분석의 기준점이다.

set_input_delay / set_output_delay — 칩 경계의 약속이다. 칩 입력 핀에 데이터가 "클럭 기준 언제 도착하는지" (input delay), 출력 핀이 "클럭 기준 언제까지 안정해야 하는지" (output delay)를 알려 준다. 칩 바깥의 보드·다른 칩과의 인터페이스 타이밍을 STA에 가져오는 통로다. 메모리의 DQ-DQS 같은 I/O 타이밍이 바로 여기서 모델링된다 — strobe 기준으로 데이터가 언제 와야/나가야 하는지를 input/output delay로 못 박는다.

set_false_path — "이 경로는 타이밍을 잡지 말라"고 STA에 알려 주는 선언이다. 왜 필요한가? 회로에는 논리적으로 **절대 활성화되지 않거나, 타이밍을 지킬 필요가 없는 경로**가 있다. 대표가 **비동기 클럭 도메인 간 경로**다. 서로 비동기인 두 클럭 사이 경로는 한 클럭 주기 안에 데이터가 도착할 의무가 없다(대신 CDC 동기화로 따로 보호한다). 또 상호 배타적인 MUX 경로, 정적 config 경로 등도 false path다. 이것 안 빼면 STA가 "도달 불가능한 위반"을 잔뜩 보고해 진짜 위반이 묻힌다.

set_multicycle_path (MCP) — "이 경로는 한 사이클이 아니라 N 사이클에 한 번만 잡히면 된다"고 알려 준다. 어떤 데이터 경로는 의도적으로 여러 사이클에 걸쳐 천천히 계산된다(예: 느린 곱셈기를 2사이클에 한 번 샘플). 이 경로에 1사이클 setup을 강요하면 거짓 위반이 난다. MCP N을 주면 setup 마감이 $N \times T_{clk}$ 로 완화된다.

여기서 면접·실무 모두에서 **결정적으로 중요한 경고**가 있다: **false path나 MCP를 잘못 걸면 진짜 위반을 숨긴다**. STA에게 "이 경로 보지 마"라고 말하는 순간, 그 경로가 실제로는 1사이클 안에 동작해야 하는데 false로 빠졌다면, 실리콘에서 그 경로가 깨져도 STA는 영원히 "통과"라고 말한다. 그래서 false path는 **구조적 근거(비동기 도메인임이 CDC로 입증됨, MUX가 정말 배타적임)**가 있을 때만 걸고, 반드시 리뷰한다. 이것이 STA와 CDC가 짝을 이루는 지점이다 — false path로 STA에서 뺀 비동기 경로는 CDC sign-off로 따로 검증해야 비로소 안전이 완성된다.

또 하나의 함정: **MCP를 setup만 완화하고 hold를 안 맞추면 hold가 깨진다**. setup을 N사이클로 늘리면 그에 짝이 되는 hold 기준도 함께 옮겨 줘야(hold MCP) 한다. 안 그러면 hold 체크가 엉뚱한 엣지를 기준으로 잡혀 거짓 위반이 나거나, 더 나쁘게는 진짜 hold가 가려진다. "MCP 걸 때 hold도 같이 본다"는 게 실무 체크리스트의 단골이다.

기억할 한 문장: SDC는 STA에게 의도를 알려 주는 언어 — create_clock(박자), input/output_delay(경계 약속), false_path(타이밍 제외), MCP(N사이클 완화). **잘못 걸면 진짜 위반을 숨긴다.** false path는 구조 근거 +CDC로, MCP는 hold까지 같이.

8. Timing closure — 위반을 없애 가는 반복 루프

지금까지의 모든 개념이 실무에서 합쳐지는 곳이 **timing closure**다. 합성(synthesis)과 배치배선(P&R)을 거친 설계에는 거의 항상 setup·hold·transition(슬루)·DRV(전기적 규칙) 위반이 남는다. closure는 이 위반들을 **모든 코너·모든 모드(MCMM, Multi-Corner Multi-Mode)**에서 $slack \geq 0$ 이 될 때까지 반복적으로 없애는 과정이다. 루프는 대략 이렇게 돈다:

- | | |
|--|--|
| 1. STA 분석 (PrimeTime, 모든 corner/mode) | |
| ↓ WNS/TNS/위반 경로 리스트 | |
| 2. 위반 분류 | |
| - setup violation (느린 경로) / hold (빠른 경로) | |
| - transition/DRV | |
| - 진짜 위반 vs SDC 문제(false/MCP 누락) | |
| ↓ | |
| 3. ECO 결정 (위반 종류에 맞는 국소 수정) | |
| - setup: 셀 upsize, VT 낮추기(VT swap),
버퍼로 리파워, 로직 단순화/재구조 | |
| - hold: 버퍼/delay cell 삽입(빠른 경로 늦추기) | |
| ↓ | |
| 4. ECO 적용 (placement·routing 최소 교란) | |
| ↓ | |
| 5. LEC로 기능 보존 확인 (ECO가 논리를 안 바꿨는지) | |
| ↓ | |
| 6. 재분석 → 1번으로 (수렴할 때까지) | |

위반 분류(2단계)가 closure의 지능이 모이는 곳이다. 같은 "위반"이라도 처방이 정반대다. setup 위반은 경로를 **빠르게**, hold 위반은 경로를 **느리게** — 5절에서 본 시소다. 게다가 "진짜 타이밍 위반"인지 "SDC가 잘못 걸려서 난 거짓 위반"인지를 가려야 한다. false path/MCP가 빠져서 난 위반을 ECO로 고치려 들면 헛수고이거나 회로를 망친다. 당신이 삼성에서 한 일 — timing/CDC/lint 결과를 통합·자동 분류해 closure 효율을 높인 것 — 이 정확히 이 분류 단계의 자동화다.

ECO(Engineering Change Order, 3·4단계). placement·routing을 크게 흔들지 않고 국소적으로 고치는 수정이다. 크게 흔들면 멀쩡하던 다른 경로가 깨져 closure가 발산하기 때문이다. setup ECO는 셀 키우기 (upsize)·VT 낮추기·버퍼 재배치·로직 단순화로 경로를 빠르게 한다. hold ECO는 버퍼/지연 셀을 끼워 빠른 경로를 느리게 한다. hold ECO가 더 까다로운 건 3절에서 본 대로 — setup을 깨지 않으면서, 거의 모든 코너에서 동시에 잡아야 하고, 위반 경로가 광범위하게 흩어져 있어서다.

LEC(Logic Equivalence Check, 5단계)가 closure 루프의 안전벨트다. ECO는 셀을 바꾸고 버퍼를 끼우는 등 netlist를 건드린다. 이때 **기능(논리)이 변하지 않았음**을 formal하게 증명해야 한다. 버퍼는 논리적으로 항등

(buffer in=out)이라 LEC를 통과해야 정상이고, 셀 size 교체도 기능은 같아야 한다. 만약 ECO 스크립트의 실수로 논리가 바뀌면 LEC가 잡아낸다. LEC는 시뮬레이션과 달리 벡터 없이 **exhaustive하게** RTL↔netlist 또는 netlist↔netlist의 논리 동치를 증명한다(Formality/Conformal). 당신이 Quick/Full EQ 이중화로 1,760만 라인 규모를 적은 인력으로 무결성 확보한 그 작업이, ECO 이후 기능 보존을 보증하는 바로 이 단계다. STA로 "타이밍이 맞다"를 보증하고, LEC로 "기능이 그대로다"를 보증해야 비로소 ECO가 안전하게 완료된다.

기억할 한 문장: closure = 분석→분류→ECO→(LEC로 기능 보존 확인)→재분석의 수렴 루프. setup ECO=빠르게(upsample/VT), hold ECO=느리게(버퍼 삽입). **STA가 타이밍을, LEC가 기능을** 동시에 지켜야 ECO가 완성된다.

9. CTS·skew 관리, 그리고 메모리의 DLL/PLL — STA가 극한으로 작동하는 무대

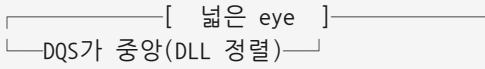
이제 STA를 메모리, 특히 HBM의 좌표에 올려놓자. 여기서 당신의 강점이 그대로 무기가 된다.

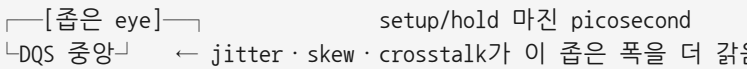
CTS(Clock Tree Synthesis)와 skew 관리. 5절에서 skew가 setup·hold에 정반대로 작용함을 봤다. 그 skew를 작게, 또는 의도적으로(useful skew) 다루는 물리 단계가 CTS다. 클럭을 칩 전역에 분배하면서 각 FF까지의 도착 시각을 균형 잡는다. skew를 줄이면 hold 마진이 안정되고(빠른 경로가 옛 값을 침범할 여지 감소), 필요하면 critical path의 capture 클럭을 살짝 늦춰 setup을 빌려 오는(useful skew) 정교한 조작도 한다. 잘 지어진 클럭 트리는 hold closure의 짐을 크게 덜어 준다.

메모리의 DLL/PLL — 내부 클럭과 외부 I/O 클럭의 정렬. 메모리 칩에는 코어 로직의 클럭과, 외부 인터페이스(DDR/HBM I/O)의 클럭이 따로 있고, 이 둘을 **정렬(align)**하는 것이 결정적이다. **DLL(Delay-Locked Loop)**은 지연을 조절해 내부에서 만든 출력 클럭/스트로브가 외부 클럭과 **위상 정렬**되도록 한다. **PLL(Phase-Locked Loop)**은 위상뿐 아니라 **주파수 합성**까지 한다(저주파 기준 클럭에서 고주파를 생성). 메모리 I/O에서 이 정렬이 왜 사느냐 죽느냐인가? **데이터(DQ)를 스트로브(DQS)로 잡는** source-synchronous 방식이기 때문이다. 데이터와 함께 보내진 스트로브로 데이터를 캡처하는데, 그 스트로브의 위상이 데이터 윈도우(data eye) 중앙에 정확히 놓여야 setup/hold 마진이 최대가 된다. DLL/PLL이 그 정렬을 책임지고, write/read leveling·training으로 미세 보정한다.

왜 DDR/HBM에서 STA가 극한으로 중요한가. 1st-principles로 풀자. setup/hold 마진은 본질적으로 **data eye의 폭** 안에서 확보되는 여유다. 그런데 속도가 올라갈수록(HBM4 per-pin 약 10GT/s — 변동/추정) 한 비트에 허용된 시간 창(unit interval)이 **picosecond 단위로 쪼그라든다**. 그 좁은 창 안에서 jitter는 eye를 더 좁히고, skew는 마진을 갉고, 2048-bit(HBM4 광폭 I/O — 변동/추정)나 되는 수많은 라인 사이의 crosstalk·skew가 동시에 마진을 위협한다. 즉 **고속·광폭 I/O는 setup/hold 마진이 가장 뻣뻣해지는 무대**이고, 그 마진을 모든 코너·모든 변이에서 보증하는 일이 곧 STA sign-off다. 채널이 2048개면 그 2048개 각각의 DQ-DQS setup/hold가 picosecond 마진으로 달혀야 하고, 하나라도 음수면 그 등급은 못 낸다. 메모리 시장에서 "더 빠른 HBM"이란 곧 "더 좁은 eye에서 마진을 달아낸 STA의 승리"다.

data eye가 속도와 함께 좁아진다 (개념도):

저속:  setup/hold 마진 넉넉

고속:  setup/hold 마진 picosecond

STA의 일 = 이 좁아진 eye 안에서 모든 코너 · 변이에 $slack \geq 0$ 보증

기억할 한 문장: CTS가 skew를 다스리고, 메모리의 DLL/PLL이 내부 클럭과 외부 I/O 클럭(DQ-DQS)을 위상 정렬한다. 속도($\approx 10\text{GT/s}$)·광폭(2048-bit) I/O에서 data eye가 picosecond로 좁아질수록 setup/hold 마진 sign-off, 즉 STA의 가치가 극한으로 커진다.

10. 면접 연결 — 30초/1분 골격과 브릿지

이제 위의 1st-principles를 **말로 꺼내는 훈련**이다. 면접관이 던지는 ★빈출 질문에 대한 30초(핵심)와 1분(전개 +브릿지) 두 버전을 준비한다. 외운 티 없이, 식이 아니라 **이유**로 말하는 게 핵심이다.

★ "setup과 hold violation의 차이는?"

30초 골격: "setup은 데이터가 다음 엣지 전에 도착해야 한다는 제약이라, 느린(max) 경로가 위협하고 주파수를 낮추면 회피됩니다. hold는 데이터가 같은 엣지 후에도 잠깐 유지돼야 한다는 제약이라, 빠른(min) 경로가 위협하고 주파수와 무관해 버퍼 삽입으로 고칩니다. 그래서 hold 위반이 더 위험합니다 — 실리콘에서 발견되면 어떤 주파수에서도 죽으니까요."

1분 전개: 위에 더해 — "slack은 required-arrival이고, setup은 arrival을 줄이거나 T_{clk} 을 키워서, hold는 arrival을 키워서(경로를 느리게) 좋아집니다. 부호가 반대죠. 저는 PrimeTime으로 이 둘을 직접 closure했고, 특히 hold는 setup을 깨지 않으면서 거의 모든 코너에서 동시에 잡아야 해서 손이 더 갔습니다. 이 경험이 HBM I/O처럼 마진이 뽀뽀한 곳에서 그대로 가치가 됩니다."

★ "hold를 주파수로 못 고치는 이유는?"

골격: "hold slack 식이 $(t_{cq} + t_{comb_min}) - t_h$ 라서 식 안에 T_{clk} 이 없기 때문입니다. hold는 launch와 capture가 같은 엣지를 공유하는 문제라, 엣지 사이 간격(주기)과 무관합니다. 같은 엣지에서 새 값이 옛 값을 너무 빨리 덮느냐의 문제이니, 주기를 늘려도 그 경합은 그대로입니다. 그래서 버퍼로 빠른 경로를 늦춰야 합니다."

★ "skew가 setup/hold에 미치는 영향은? (부호 반대)"

골격: "skew를 capture클럭-launch클럭 도착 차로 정의하면, capture가 늦게 올 때($skew > 0$) setup엔 +로 들어가 유리합니다 — 데이터에게 마감을 미뤄 주니까요. 그런데 hold엔 -로 들어가 불리합니다 — capture의 포획 시점이 뒤로 밀려 hold 윈도우가 넓어지고, launch의 빠른 새 값이 그걸 침범하기 쉬워집니다. 같은 skew가 정반대 부호로 작용하는 게 핵심입니다. 그래서 CTS로 skew를 다스리는 게 hold closure의 사전 작업입니다."

★ "OCV/corner는 왜 보나?"

골격: "양산되는 칩은 공정 분포(SS~FF)가 있고 동작 전압·온도도 넓게 변하는데, 그 어떤 조합에서도 동작해야 하기 때문입니다. STA는 그 분포의 양 극단을 코너로 잡아 모두 $\text{slack} \geq 0$ 을 봅니다. **setup은 셀이 느려지는 slow 코너, hold는 빨라지는 fast 코너**가 worst죠. OCV는 거기에 더해 **한 칩 안의 국소 변이**까지 derate로 끌어안는 것이고, 단순 OCV의 과도한 비관은 AOCV/POCV로 현실화합니다. 메모리는 동작 범위가 넓어 코너가 많고 fast 코너의 hold가 까다롭습니다."

브릿지 한 문장 (모든 STA 답변의 마무리)

"삼성에서 PrimeTime으로 Peri/로직의 timing closure를 직접 돌려 sign-off 감을 익혔고, 이 setup/hold·skew·코너 사고가 HBM 10GT/s·2048-bit I/O의 DQ-DQS 마진 sign-off로 그대로 확장됩니다. 메모리를 몰라서가 아니라, 제가 하던 일이 바로 그 마진을 닫는 일입니다."

이 브릿지의 힘은 "용어가 같다"가 아니라 **"문제가 같다"**에 있다. timing closure에서 마진을 picosecond 단위로 닫아 본 사람만이 HBM I/O의 뻑뻑함을 몸으로 안다. 면접관에게 줄 메시지는 명료하다 — **STA는 당신의 갭이 아니라 당신의 무기다.**

11. 스스로 점검 — 면접 전 자가 구술 테스트

아래 질문에 **식이 아니라 이유로**, 막힘없이 소리 내어 답할 수 있어야 한다. 하나라도 더듬으면 해당 절로 돌아가 1st-principles를 다시 세운다.

1. STA가 시뮬레이션을 대체하는 이유 세 가지(벡터 불필요·exhaustive·빠름)를 각각 **왜** 그런지 설명할 수 있는가? sign-off가 무슨 뜻인지 한 문장으로 말할 수 있는가?
2. 동기 설계의 "약속"을 launch→logic→capture로 그리고, setup과 hold가 **왜 둘 다** 필요한지(플립플롭 내부 동작으로) 설명할 수 있는가?
3. setup slack 식을 arrival/required로 분해하고, **주파수를 낮추면 왜 풀리는지**를 "데이터에게 시간을 더 준다"로 설명할 수 있는가?
4. hold slack 식에 **T_{clk}이 없는 이유**를 "같은 엠티 문제"로 설명하고, 그래서 주파수 무관·chip dead임을 말할 수 있는가?
5. hold를 버퍼로 고칠 때 **왜 까다로운지**(setup 동시 악화, 다중 코너, 위반 분산) 세 가지를 댈 수 있는가?
6. slack 부호의 비대칭(경로를 빠르게→setup↑ hold↓)을 말하고, WNS와 TNS를 **왜 둘 다** 보는지 설명할 수 있는가?
7. clock skew를 부호와 함께 정의하고, **같은 skew가 setup엔 +, hold엔 -**로 작용하는 이유를 capture 엠티 이동으로 설명할 수 있는가?
8. jitter와 skew의 차이(동적 vs 정적), uncertainty가 setup엔 -·hold엔 +로 들어가는 pessimism 원리를 설명할 수 있는가? PVT corner에서 **setup=slow, hold=fast**인 이유를 셀 속도로 설명할 수 있는가?
9. false path와 MCP를 **잘못 걸면 무슨 일**이 나는지(진짜 위반을 숨김), false path가 CDC와 짝인 이유, MCP에 hold도 같이 봐야 하는 이유를 말할 수 있는가?
10. closure 루프에서 **LEC가 왜 필요한지**(ECO 후 기능 보존), STA(타이밍)와 LEC(기능)의 역할 분담을 설명할 수 있는가?

11. 메모리 DLL/PLL이 내부·외부 클럭을 정렬하는 이유와, 고속·광폭 I/O에서 data eye가 좁아져 STA가 극한으로 **중요해지는** 1st-principles 사슬을 끝까지 말할 수 있는가?
12. 위 모든 답 끝에 **"내가 하던 timing closure = HBM I/O 마진 sign-off"** 브릿지를 자연스럽게 붙일 수 있는가?

답는 말: STA의 모든 공식은 결국 **"한 클럭 안에 데이터가 안전하게 한 칸 전진한다"**는 동기 설계의 약속 하나에서 연역된다. setup은 그 약속의 "늦지 마라" 절반, hold는 "너무 빨리 떠나지 마라" 절반이다. skew·jitter·OCV·corner는 그 약속을 **현실의 불확실성 속에서도** 지키게 하는 보수적 안전장치이고, SDC는 그 약속에 의도를 입히는 언어이며, closure는 그 약속을 모든 조건에서 닫아 가는 노동이다. 이 사슬을 1st-principles로 붙들면, HBM의 picosecond 마진 앞에서도 흔들리지 않는다. 당신은 이미 그 마진을 닫아 본 사람이다.